K. J. Somaiya College of Engineering, Mumbai-77
(A Constituent College of Somaiya Vidyavihar University)
Department of Computer Engineering

Batch: b2	**Roll No.: 16010124107**
Experiment No.3	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

Title: Study, Implementation, and Comparative Analysis of Merge Sort and Quick Sort.

Objective: To learn the divide and conquer strategy of solving the problems of different types

CO to be achieved:

CO 2 Describe various algorithm design strategies to solve different problems and analyze Complexity.

Books/ Journals/ Websites referred:

1. **Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen, C.E.Leiserson, R.L.Rivest and C.Stein," Introduction to algorithms", 2nd Edition, MIT press/McGraw Hill, 2001**
3. **http://en.wikipedia.org/wiki/Quicksort**
4. **https://www.cs.auckland.ac.nz/~jmor159/PLDS210/qsort.html**
5. **http://www.cs.rochester.edu/~gildea/csc282/slides/C07-quicksort.pdf**
6. **http://www.sorting-algorithms.com/quick-sort**
7. **http://www.cse.ust.hk/~dekai/271/notes/L01a/quickSort.pdf**
8. **http://en.wikipedia.org/wiki/Merge_sort**
9. **http://www.personal.kent.edu/~rmuhamma/Algorithms/MyAlgorithms/Sorting/mergeSort.htm**
10. **http://www.sorting-algorithms.com/merge-sort**
11. **http://www.princeton.edu/~achaney/tmve/wiki100k/docs/Merge_sort.html**

Pre Lab/ Prior Concepts:

Data structures, various sorting techniques

Historical Profile:

Quicksort and merge sort are divide-and-conquer sorting algorithms in which division is dynamically carried out. They are one the most efficient sorting algorithms.

New Concepts to be learned:
Number of comparisons, Application of algorithmic design strategy to any problem, Classical problem solving vs Divide-and-Conquer problem solving.

Algorithm Recursive Quick Sort:

void quicksort(Integer A[ ], Integer left, Integer right)
//sorts A[left.. right] by using partition() to partition A[left.. right], and then //calling itself // twice to sort the two subarrays.
{ **IF** (left < right) then
{ q = partition(A, left, right);
quicksort(A, left, q–1);
quicksort(A, q+1, right);
}
}

Integer *partition(integer A*T[], Integer *left*, Integer *right*)
//This function re-arranges *A*[*left*..*right*] and finds and returns an integer *q*, such that *A*[*left*], ..., //*A*[*q*–1] <~ *pivot*, *A*[q] = p*ivot*, *A*[*q*+1], ..., *A*[*right*] > *pivot*, where *pivot* is the first element of //a[left…right], before partitioning.
{
pivot = A[left]; lo = left+1; hi = right;
WHILE (lo ≤ hi)
{ **WHILE** (A[hi] > pivot) hi = hi – 1;
WHILE (lo ≤ hi and A[lo] <~pivot) lo = lo + 1;
IF (lo ≤ hi) then swap(A[lo], A[hi]);
}
swap(pivot, A[hi]);
RETURN hi;
}

The space complexity of Quick Sort:

Quick Sort uses recursion stack space. Average depth is log n. Worst case depth is n. Average space is O(log n). The worst case space is O(n).

Derivation of best case and worst-case time complexity (Quick Sort) Algorithm Merge Sort
code:

```
quicksort(int l, int h){
    if(l<h){
        j = partition(l,h);
        quicksort(l,j);
        quicksort(j+1,h);
    }
}
```

Ashmera Hasan
16010124107

Quick Sort

Partition $\rightarrow \theta(n)$

Recurrence =

$$T(n) = T(k) + T(n-k-1) + \theta(n)$$

Best case: $T(n) = 2T(n/2) + \theta(n)$

using master thm,

$a = 2 \quad b = 2 \quad f(n) = \theta(n)$

$n^{\log_b a} = n^{\log 2} = n$

$\therefore \quad T(n) = \theta(n \log n)$

Worst case:

$$T(n) = T(n-1) + \theta(n)$$

$$T(n-1) = T(n-2) + \theta(n-1)$$

$$T(n) = T(n-2) + \theta(n-1) + \theta(n)$$

$$T(n) = T(n-k) + \theta(n)$$

$$= T(0) + \frac{n(n+1)}{2} \qquad \text{when } n = k$$

$$= O(n^2)$$

Avg case:

$$T(n) = \frac{1}{n} \sum_{k=0}^{n-1} [T(k) + T(n-k+1)] + \theta(n)$$

By symmetry:

$$T(n) = \frac{2}{n} \sum_{k=0}^{n-1} T(k) + \theta(n)$$

$$nT(n) = \frac{2}{\cancel{n}} \cancel{n} \sum_{k=0}^{n-1} T(k) + n\theta(n)$$

$$(n-1)T(n-1) = 2 \sum_{k=0}^{n-2} T(k) + c(n-1)$$

$$T(n) = n \log n.$$

MERGE-SORT *(A, p, r)*

// To sort the entire sequence A[1… n], make the initial call to the procedure MERGE-SORT (*A*, //1, *n*). Array *A* and indices *p*, *q*, *r* such that $p \leq q \leq$ `r and sub array` *A*[*p* .. *q*] is sorted and sub array //*A*[*q* + 1 .. *r*] is sorted. By restrictions on *p*, *q*, *r*, neither sub array is empty.
//OUTPUT: The two sub arrays are merged into a single sorted sub array in *A*[*p* .. *r*].

IF $p < r$ // Check for base case
  THEN q = FLOOR [$(p + r)/2$] // Divide step

```
(A, p, q)                    //Conquer step.
        MERGE (A, q + 1, r)          // Conquer step.
        MERGE (A, p, q, r)           // Conquer step.


MERGE (A, p, q, r)
{
    n1 ← q − p + 1
    n2 ← r − q
    Create arrays L[1 ... n1 + 1] and R[1 . . n2 + 1]
   FOR i ← 1 TO n1
        DO L[i] ← A[p + i − 1]
      FOR j ← 1 TO n2
        DO R[j] ← A[q + j ]
    L[n1 + 1] ← ∞
    R[n2 + 1] ← ∞
   i ← 1
   j ← 1
   FOR k ← p TO r
      DO IF L[i ] ≤ R[ j]
           THEN A[k] ← L[i]
                i ← i + 1
           ELSE A[k] ← R[j]
                j ← j + 1

}
```

The space complexity of Merge sort:

The space complexity of merge sort is O(n) as it involves creating an auxiliary array that stores the preliminary values before merging.

Derivation of best case and worst-case time complexity (Merge Sort)

Ashmeen
16010124107

Merge sort

→ divide in 2 parts $= O(1)$
→ recursively sort 2 halves $= 2T(n/2)$
→ Merge those halves $= O(n)$

Recurrence is $T(n) = 2T(n/2) + cn$

$a = 2 \quad b = 2 \quad f(n) = \theta(n)$

$n^{\log_b a} = n$

$T(n) = \theta(n \log n)$

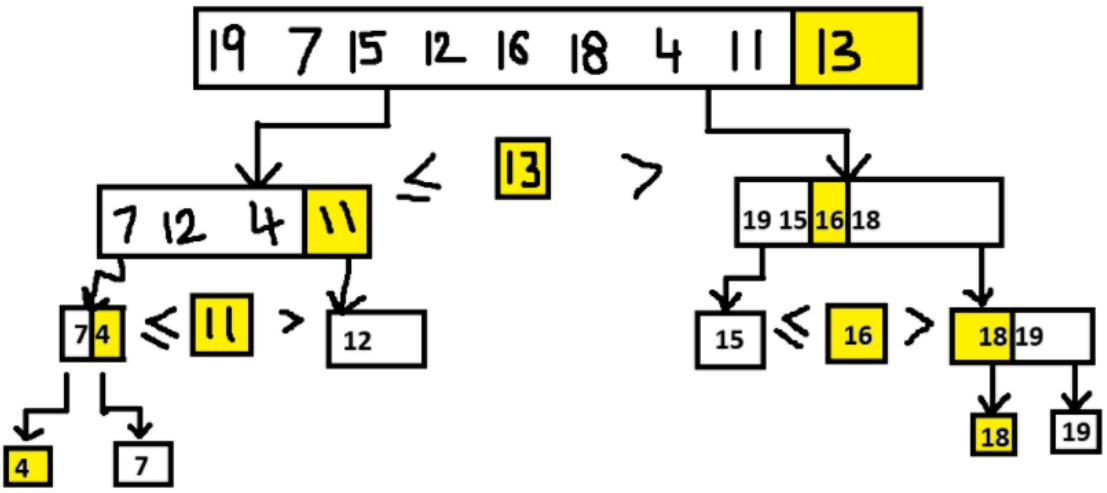
∴ for best, worst, and average, time complexity is $O(n \log n)$ as merge sort always divides evenly and merges in linear time.

The best case and worst case complexity of merge sort is to O(n log n) since splitting the array takes constant time at each level of recursion, irrespective of the initial state of the array.

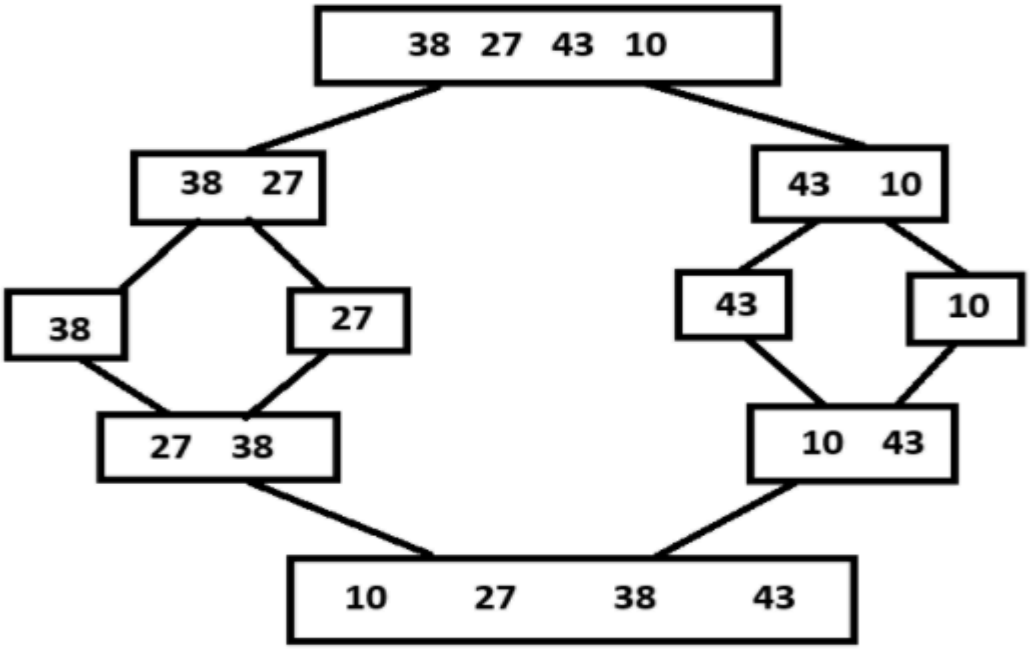
```
MERGE-SORT(A, left, right)

    if left < right

        mid ← (left + right) / 2

        MERGE-SORT(A, left, mid)

        MERGE-SORT(A, mid + 1, right)

        MERGE(A, left, mid, right)
```

Example for quicksort/Merge tree for merge sort:

QUICK SORT



MERGE SORT



CONCLUSION:

Merge Sort and Quick Sort use divide and conquer effectively. Merge Sort guarantees O(n log n) time. Quick Sort is faster in practice. Choice depends on space and input.